

Reproducing Multi-Agent System Failures: ChatDev 1.0 and Magentic-One under gpt-5.4-mini

MAS memory research — 2-page summary — June 10, 2026

Question. MAST (“Why Do Multi-Agent LLM Systems Fail?”, arXiv:2503.13657) groups multi-agent failures into three categories; category 2, *inter-agent misalignment*, covers failures of communication between agents — information withheld, input ignored, conversations reset. We ask whether these are *capability* failures (they fade with a stronger model) or *structural* failures (built into how the systems communicate and remember). This report covers step one: re-run a sample of MAST’s tasks with a two-generation-newer model and re-label the runs with a better judge.

1 Dataset construction

The 30-task dataset was deliberately selected to maximize the chance of observing inter-agent misalignment; it is not a representative sample, and absolute failure rates should not be read as benchmark scores. MAST’s public per-trace labels turned out to be unusable (a join error assigns labels by row position), so we rebuilt ground truth: ChatDev human annotations recovered from the MAST repo’s git history, and Magentic-One outcomes computed against the GAIA gold answers. We then read each candidate’s *original GPT-4o trace* and kept it only if the original failure involved agent-to-agent communication (rather than one agent simply getting the answer wrong). Final set: 15 ChatDev game-building tasks and 15 Magentic-One GAIA questions, each mixing high/medium/low-likelihood cases plus originally-*solved* controls (3 and 2 respectively).

2 The two architectures

ChatDev 1.0 simulates a software company: a fixed “waterfall” of phases (requirements → coding → three review rounds → testing → docs), each phase a dialogue between two role-prompted copies of the model (CEO, Programmer, Reviewer, Tester). What passes from one phase to the next is *only the project’s code files* — and those files are created indirectly: when the Programmer “writes code”, it types it into a chat message as filename-tagged code blocks, and after the message a parser extracts each block and saves it to the named file. Two details are load-bearing: (i) each reply has a hard length cap (4096 tokens minus the prompt), so a long file gets cut off mid-line, and nothing checks for that; (ii) the parser only saves properly closed code blocks — a cut-off block is *silently skipped*. So an agent can “write a file”, see no error, and the file never comes to exist. **Magentic-One** is a hub-and-spoke system for web research: an Orchestrator holds a “ledger” (collected facts + current plan), delegates one step at a time to four specialists (a screenshot-based WebSurfer, FileSurfer, Coder, ComputerTerminal), and when progress stalls it rewrites the ledger from its own summary of history — a built-in reset.

Reproduction setup. Both systems run their official, unmodified code (ChatDev v1.1.6; Magentic-One via the MAST authors’ own harness, AutoGen 0.4.8). The single change is the model behind the API: a local proxy transparently serves **gpt-5.4-mini** where the systems believe they are calling GPT-4o (tool calls, JSON mode, and screenshot vision verified working). Deviations: code runs on the host rather than in Docker, and some GAIA answers have drifted since 2024 because the live web changed.

3 Outcomes vs. the original GPT-4o runs

Magentic-One: 2/15 solved originally → 3/15 now (one failure flipped to pass, no regressions, both controls still pass). ChatDev: 3/15 → 5/15 (two flips; all three controls still pass; caveat: original grades are human annotations, ours come from the judge). **The selected failure set largely survives the model upgrade**, which is what the structural hypothesis predicts.

4 MAST taxonomy statistics

Every trace is labeled by a two-tier gpt-5.5 judge: tier 1 reads the full trace with *no taxonomy* and reports anything problematic in free form; tier 2 then assigns the 14 MAST modes, and every flag must carry a verbatim quote from the trace or it is dropped. Calibrating against the human-annotated traces: modes where an agent visibly does the wrong thing (2.3–2.6, 3.x) are detected reliably — including 2.4 and 2.5, which MAST’s own o1-based judge had emitted *zero* times — while modes defined by an *absence* of action (2.1, 2.2) are caught only about half the time, so those two rows below are lower bounds.

Mode	Name	ChatDev (n=15)	Magentic-One (n=15)
2.1	Conversation Reset	7	4
2.2	Fail to Ask for Clarification	5	8
2.3	Task Derailment	6	8
2.4	Information Withholding	15	4
2.5	Ignored Other Agent’s Input	15	13
2.6	Action-Reasoning Mismatch	12	13
3.1–3.3	Verification failures	13/13/13	13/13/13

The two systems show *opposite* category-2 profiles, each matching its architecture. ChatDev flags Information Withholding in all 15 runs — including the three that produced working software — because its code-persistence channel loses files in every run; whether the task succeeds depends only on *which* files are lost. Magentic-One shows little withholding (the hub sees every message, so information rarely fails to arrive) but pervasive ignored input (the hub or a specialist receives information and doesn’t act on it). That incidence tracks topology rather than model is the core evidence for the structural reading.

5 Open-ended judge trends

The taxonomy-free tier produced ~650 findings; three mechanism chains dominate. (1) *ChatDev’s lossy persistence chain*: code truncated → file silently dropped → reviewer repeats the identical complaint → tester approves anyway (the loss step appears in the first case below; full chain in the DouDizhuPoker brief of `main.pdf`). (2) *ChatDev error masking*: when a missing file finally crashes the build, the Programmer patches *around* the crash (try/except fallbacks) instead of regenerating the file it already wrote once. (3) *Magentic-One unsupported finals*: the Orchestrator answers from memory or partial evidence instead of completing its own plan (7 traces), and ledger rewrites discard progress already made.

6 Four case briefs (chosen for inter-agent misalignment relevance)

ChatDev: Gomoku — *succeeded*, yet the communication channel failed

Gomoku (five-in-a-row) is a *control*: the original GPT-4o run also solved it, and ours does too. Even so, the file-saving channel of §2 failed mid-run: the Programmer produced three correct files, but the parser attached the wrong block to one of them, saving the board-logic file `board.py` with the literal one-line content “`python main.py`” (a shell command from the same message). A correct implementation was produced and destroyed in transit — Information Withholding (2.4) without any agent deciding to withhold anything. Recovery succeeded here: the Reviewer flagged the garbage file, and the regenerated `board.py` was short enough to survive, so the finished game works. This is why 2.4 appears in all 15 ChatDev runs including the successful ones: the loss mechanism fires regardless, and the task outcome depends only on whether the lost piece happens to be recoverable in the remaining rounds.

ChatDev: Wordle — *failed*; behavioral misalignment, no truncation involved

The task: a terminal Wordle clone (guess a daily five-letter word in six tries, with colored feedback). In contrast to the case above, nothing here is lost in transit — each failure is an agent’s own decision, and the run still fails. (1) The prompt explicitly demands a terminal game, but ChatDev’s pipeline injects a boilerplate instruction that the software “should be equipped with graphical user interface”; facing two contradictory requirements, the Programmer silently picked one and moved on rather than asking which was meant (2.2 Fail to Ask for Clarification — the same mode the MAST human annotators marked on the original GPT-4o run of this task). (2) The Reviewer gave a concrete, correct instruction: the word list contains the six-letter word “debate”, remove it. The Programmer went through the modification round, yet the shipped code still contains “debate” — feedback received and not acted on (2.5), leaving the delivered game in violation of the five-letter spec. (3) Another review round declared the duplicate-letter scoring the highest-priority bug while recommending, as the fix, the exact code already present (2.6 Action–Reasoning Mismatch: the reviewer’s diagnosis and proposal contradict each other), sending the Programmer to churn on a non-bug. (4) The automated test ran the interactive game with no keyboard attached and crashed on `EOFError`; the “fix” made the program exit immediately whenever input is unavailable, after which the test passed *because gameplay was never reached* (3.3): the system made the test pass rather than the game work.

Magentic-One 5a0c1adf — *failed*; correct plan, specialist doesn’t execute

A GAIA web-research question: among winners of the Malko conducting competition after 1977, find the one whose recorded nationality is a country that no longer exists. The Orchestrator planned correctly and gave WebSurfer specific, correct instructions — open the winners archive on the competition’s official site. WebSurfer instead kept re-running general web searches that hit CAPTCHAs and dead ends, and never opened the archive page it was pointed to: the instruction arrived and was ignored (2.5). After the loop stalled, the Orchestrator broke its own stated rule (“do not guess before filtering all winners”) and submitted an unsupported guess (2.6). The information flow worked; execution and verification did not.

Magentic-One 72e110e7 — *failed*; reset loop replaces recovery

A bibliographic lookup in BASE, an academic search engine. The needed page rendered blank in the browser — a recoverable tool failure. WebSurfer responded only by waiting, despite explicit instructions to inspect the page source or try another route (2.5). The Orchestrator then fell into Magentic-One’s built-in reset behavior: it kept rewriting its fact sheet and plan from scratch (2.1 Conversation Reset) — and although the rewritten plans correctly named the fix (use an archived copy of the page), it never delegated that step to any agent (2.4: the hub kept its own recovery plan to itself) before answering “unknown”. A minor tool hiccup became task failure because the recovery mechanism resets instead of acting. (*All 30 case briefs: main.pdf.*)